

WEEK 15 · FINAL

成本性能与上线收官

Capstone 课程毕业——让 AI 系统能“长期稳定跑”，同时给你一份能向陌生人证明“我能做”的作品

01 Cost

成本模型

5 类拆账 + 5 抓手

02 Resilience

韧性兜底

5 级降级 + 多级缓存

03 SLO

SLO 契约

6 大 SLO + 错误预算

04 Runbook

应急体系

Wheel + Agentic SRE

05 Capstone

毕业典礼

15 周收口 + 演进路线

本周划重点

前 14 周,我们把一个能上线、能过监管的 AI 系统建出来了。可上线之后最容易爆雷的,往往不是技术,是账单和半夜的故障。这一周补齐“长期稳定跑”这最后一块:成本可控、异常不白屏、质量有契约、故障有流程。最后,再把 15 周收口成一份你能拿去面试、拿去答辩的 Capstone 作品。

LESSON 01 · COST MODEL

精细化成本模型：让 AI 系统的"账单"不再是"财务刺客"

前 14 周建好了能上线、能监管的系统——上线 3 个月后最容易爆雷的，是账单

真实场景 · 上线三个月，CFO 拿着账单找上门

我见过一个团队，系统跑得挺好，直到 CFO 拿着一张账单杀过来："上个月 LLM API 花了 12 万，pgvector 实例月 1.8 万，Embedding 跑批还有 6 千，这钱到底花哪了？" 团队当场答不上来——因为他们对成本的全部认知，就是"看 LLM 那张月账单"。这就跟传统团队只看 AWS 月账单一样，完全丧失了工程化优化的可能。这一讲我教你：怎么把 AI 成本做成可拆账、可归因、可优化的工程对象——让财务可控不再指望"模型变便宜"，而是靠你的工程优化，一个季度稳稳降 20% 到 40%。

咱们这节聊 4 件事：

为什么"看 LLM 月账单"是 AI 团队最常见的成本管理反模式

AI 系统的 5 类成本怎么拆、每查询成本怎么算——不同范式差 5 到 15 倍

5 大优化抓手按 ROI 排序——语义缓存为什么是回本最快的那个

2026 成本利器：LiteLLM 网关 + prompt caching + Batch API + 模型价格崩塌



COST IS AN ENGINEERING OBJECT

AI 成本不是"账单数字"——是可拆账、可归因、可优化的工程对象

只看月账单，就像传统团队只看 AWS 账单，彻底放弃了工程化的可能

拆账

- 按 Embedding/检索/生成/存储/缓存分项
- 知道钱花在哪一段
- 不是一个笼统总数

归因

- 每查询能算出"花在哪"
- 按 release_id / span 归因
- Week 14 manifest 派上用场

监控

- 成本进 Week 12 dashboard
- 和质量同等重要
- 写 PR 时就看到成本影响

优化

- 按 ROI 排序做, 不拍脑袋
- 哪贵削哪 = 削错地方
- 数据驱动和优化排序

COST PER QUERY

"每查询成本"是 AI 成本最重要的单位指标——不同范式差 5 到 15 倍

这也是 Week 11 "按题型路由"的成本视角：用对范式直接省 60%

场景	主要消耗	cost / 1k queries	大头
纯向量 RAG	向量召回 + LLM 生成	\$1.5-3	60% 生成
混合 RAG (W8)	+ BM25 + Cross-Encoder rerank	\$2.5-5	50% 生成 + 20% rerank
Agent 调工具	+ 多 LLM 调用 + tool 执行	\$5-15	70% LLM 多轮
GraphRAG Global	+ 社区检测 + 摘要 + 二次生成	\$8-25	60% 多次 LLM
HITL 触发	+ 通知 + 人工等待 + 二次生成	\$15-50	人工时间占大头

关键判断

你盯着这张表看一眼就明白:成本失控往往不是"单价贵",是"范式用错"——把一个 FAQ 硬塞进 GraphRAG Global,成本能差十倍。所以 Week 13 那个"题型路由"在这儿有了新的意义:它不光提质量,更是最大的省钱手段。用对范式,比换便宜模型省得多。

OPTIMIZATION LEVERS

5 大成本优化抓手——按 ROI 排序，别"哪里贵削哪里"

ROI = 节省成本 / 工程投入，从高 ROI 开始做

抓手	具体做法	节省	ROI
Cache 高频查询	识别 top-100 高频 query 缓存	20-40%	★★★★★
题型路由	简单问题不用图 / 不用长文	15-30%	★★★★★
Embedding 复用	同 chunk 不重复 embed	5-15%	★★★★
模型降级	简单问题用 mini / haiku	20-50% 生成	★★★★
Batch 推理	Embedding / Rerank 批量	30% Compute	★★★

关键判断

前两个抓手(缓存 + 题型路由)投入小、回报大,几乎所有团队上线 3 个月内必做,回本周期通常不到一周。我的经验:别一上来就搞那些花哨的优化,先把这俩做扎实,成本先砍三成,再谈别的。优化和治理一样,也要讲 ROI——把工程时间花在回本最快的地方。



SEMANTIC CACHE

语义级缓存——ROI 最高的抓手，命中 35% 流量、3 天回本

不只字面匹配，"几乎一样"的问题也能命中；用高阈值防误命中

```
# services/cache/semantic_cache.py
class SemanticCache:
    """语义级缓存——字面 O(1) + 语义 O(log n) 双存"""
    similarity_threshold = 0.92 # 高阈值,防误命中导致答错

    def lookup(self, query, tenant, release_id):
        # 1. 字面匹配 O(1)
        key = f"sem:{tenant}:{release_id}:{hash(query)}"
        if self.r.exists(key):
            return json.loads(self.r.get(key))
        # 2. 语义匹配 O(log n)——向量检索最近邻
        q_emb = self.embedder.embed(query)
        near = self.vec_index.search(q_emb, top_k=1,
            filters={"tenant": tenant, "release_id": release_id})
        if near and near[0]["score"] > self.similarity_threshold:
            return json.loads(self.r.get(near[0]["key"]))
        return None # cache miss

# 实战指标 (100k DAU, top-100 query 占 60% 流量)
# 缓存命中率: 35% | 月成本节省: ~$15k
# 工程投入: 2 工程师周 | ROI 回收期: 3 天
```

老司机解读

similarity_threshold=0.92 高阈值 · 只缓存"几乎一样"的 query,防误命中答错

release_id 绑定 · 不同 release 缓存隔离,上新版自动失效旧缓存(接 Week 14)

tenant 隔离 · 多租户不互相缓存,合规要求

字面 + 语义双存 · 大部分流量走 O(1),语义兜底捞"换了说法的同一个问题"

ROI 3 天回本 · 月省约 \$15k、年化近 \$180k,这是 AI 工程里投入产出比最高的一件事,别的优化都排它后面

COST LEVERS · 2026

2026 新利器： LiteLLM 网关 + prompt caching + Batch——顺带模型价格在崩

这几个 2026 的东西，能在你上面那套优化之外再砍一大截

利器	做什么	省多少
LiteLLM 网关	统一 100+ 模型 + 硬预算上限 + 成本归因	预算不超支,单点管控所有花费
Prompt Caching	缓存 prompt 静态部分(系统提示/长上下文)	缓存 token 省约 90%,长上下文场景最爽
Batch API	Embedding / 离线评测走批量	约 -50%,凡是不赶时间的都该走它
模型价格崩塌	DeepSeek-V3 约 \$0.27/百万tok vs gpt-4o \$2.5	同等能力单价一年掉一个数量级

老司机解读

2026 我强烈建议你上 LiteLLM 当统一网关——它把"调 100 多家模型"收成一个 OpenAI 格式的口子,还自带硬预算上限(超了直接拦)、tag 级成本归因、自动注入 prompt caching,业务代码一个字不用改。配上 prompt caching(系统提示和长上下文缓存起来,省约九成 token 钱)+ Batch API(离线活儿一律 -50%),再叠加模型价格这两年的雪崩(gpt-4o-mini 约 \$0.15/百万 token,比两年前主力模型便宜一两个数量级),你会发现:2024 那套"AI 太贵"的焦虑,2026 大半是工程没做到位,不是模型贵。

COST DASHBOARD

成本监控面板——Week 12 升级版，写 PR 时就看到成本影响

把成本接进和质量同一块面板，建立"成本和质量同等重要"的文化

```
# observability/dashboards/cost.py
# Panel 1 · 日成本趋势
"Total Cost Daily": "sum(cost_usd) by day"
"Cost by Category": "sum(cost_usd) by day, category" # llm/embed/storage
"Cost per 1k Queries": "Total / (QPS * 86400 / 1000)" # 真正的单位成本
# Panel 2 · Top 花钱大户
"Top spans by cost": "sum(cost_usd) by span_name limit 10"
"Top release by cost": "sum(cost_usd) by release_id limit 10" # 接 Week 14
# 告警 (Week 12 升级)
alerts:
- name: daily_cost_burn # 日成本超 7 天均值 30%
  expr: sum(cost_usd_daily) > 1.3 * avg_7d
  severity: P2
- name: monthly_budget_burn_fast # 月预算烧太快,月底会超支
  expr: monthly_spend/monthly_budget > (day/days_in_month) * 1.3
  severity: P1
  notify: [pagerduty, slack#data-cost-alerts]
```

老司机解读

Cost per 1k Queries · 这才是 AI 系统该看的"单位成本",能直接跟 SaaS 业务单价对比

Cost by Category · 一眼看出哪类成本最高,优化方向清晰

Top release by cost · Week 14 的 release_id 在成本视角发挥价值,哪个版本贵一目了然

monthly_budget_burn_fast · Week 12 那套 burn rate,搬到预算上,月底超支提前预警

成本告警和质量告警放同一个 Slack 频道 · 让团队把"成本"当"质量"一样认真

INDUSTRY 2026

AI FinOps——成本管理已从"看月账单"升级为完整工程学科

工具链成熟、ROI 清晰、优化路径已知，就看你做不做

来源	做法	给你的启示
LiteLLM	统一网关 + 预算上限 + 成本归因	2026 首选,单点管住所有模型花费
OpenAI / Anthropic	Prompt Caching + Batch API	缓存省 ~90%、批量 -50%,必用
Langfuse / Helicone	trace 级成本归因	每条 trace 自带 cost,接 Week 12
FinOps Foundation	Inform / Optimize / Operate 三阶段	把云 FinOps 框架搬到 AI

老司机说

给你一个 2026 的判断:AI 成本管理已经是一门成熟学科了,不是玄学。以前大家焦虑"大模型太贵",现在真相是——贵,往往是工程没做到位:没缓存、没路由、没走批量、没上网关管预算。你把这一节的抓手 + LiteLLM + prompt caching 都用上,同样的业务量,成本砍一半很正常。这也是为什么面试时"你怎么控 AI 成本"这个问题,能一下子把有经验的和只会调 API 的区分开。

LESSON 02 · GRACEFUL DEGRADATION

韧性与兜底：让 AI 系统在异常下"有回声，不白屏"

AI 系统上线后必然遇到：OpenAI 429、pgvector OOM、上游维护、流量 10 倍突增

真实场景 · 一句"服务异常请稍后再试"，5 分钟流失 30% 客户

我印象最深的一次事故:OpenAI 突然大面积 429, 我们的 RAG 服务没做韧性, 直接把异常抛给了用户——满屏"服务异常, 请稍后再试"。你猜结果? 监控上眼睁睁看着, 5 分钟里在线用户掉了三成, 客服电话被打爆——一个中等体量 C 端服务, 这种 P0 每分钟损失常在 \$500 到 \$5,000, 几分钟就是几万块打水漂。而隔壁做了韧性的团队, 同一时刻自动降级到 BM25 检索 + 缓存兜底, 用户压根没感觉到出事。这就是差距。这一讲我带你把它做出来: 让 AI 系统在最差情况下, 也能"保住基本功能、有回声不白屏"。

咱们这节聊 4 件事:

为什么"重试 3 次"根本不是韧性——真正的韧性是降级路径

韧性的本质: 按服务等级"保住核心、放弃边缘", 不是"全部正常"

5 级降级链 + 4 级缓存 + 令牌桶限流——三件套怎么组合

2026 工具: LiteLLM router 做模型 fallback, 以及一个要纠正的旧认知(Hystrix)

KEEP CORE, DROP EDGE

韧性的本质是"按服务等级保留功能"——不是"全部正常"

传统软件追求"永远可用+永远快", AI 系统做不到——单次 LLM 调用 30s timeout 是常态

P0 · 永远保住

- "有回声不白屏"
- 不泄露 PII
- 这是红线,任何情况都保
- 掉了 = 事故

P1 · 尽量保住

- 答案基本正确
- 有引用
- 质量目标,尽力
- 可短暂降级

P2 · 主动降级

- BM25 替代向量
- mini 替代 gpt-4o
- 牺牲质量换可用
- 用户能用就行

P3 · 果断拒绝

- 复杂归纳类转人工
- 别硬扛
- 放弃边缘功能
- 保住核心资源

FIVE-LEVEL DEGRADATION

5 级降级链——从全功能到最小可用，必须显式声明 + 自动触发

不能等 oncall 来手动切；Week 10 的 fallback chain 在这里升级成"系统级"

级别	降级动作	触发条件	用户体验
L0 · 全功能	gpt-4o + Hybrid + Graph + Rerank	正常	最佳质量
L1 · 轻度	sonnet 替代 gpt-4o	gpt-4o 排队 / 5s+ 延迟	质量略降,仍优秀
L2 · 中度	关 graph,全 vector	Graph 服务慢 / 错	不能跨文档归纳
L3 · 重度	BM25 替代 vector + mini 模型	pgvector 挂 / Embedding 慢	准确度降,能用
L4 · 兜底	返回缓存命中或引导人工	所有路径都挂	不白屏 + 转人工

关键判断

这 5 级的精髓,是每一级都"预先声明 + 自动触发",绝不能等半夜 oncall 爬起来手动切——那时候黄花菜都凉了。关键是最后那个 L4:哪怕天塌了、所有链路都挂,也要给用户一句"服务繁忙,已为您预约人工",而不是甩一个 500 白屏。用户对 AI 的信任本来就脆,一次白屏可能就再也不来了。

RESILIENT RAG · CODE

分层韧性的实现——每一级都仍然返回 answer，不抛异常给用户

熔断+ 逐级降级，降级事件自动进 Week 12 告警

```
# services/rag/resilient_serve.py
from circuitbreaker import circuit

class ResilientRAGService:
    @circuit(failure_threshold=5)      # 同类错 5 次自动熔断
    async def serve(self, query, tenant) -> dict:
        try:    return await self._l0_full(query, tenant)    # L0 全功能
        except (LLMTimeoutError, LLMRateLimitError):
            metrics.degradation_event("L0_to_L1")
        try:    return await self._l1_alt_model(query, tenant) # L1 换模型
        except (LLMError, GraphServiceError):
            metrics.degradation_event("L1_to_L2")
        try:    return await self._l2_no_graph(query, tenant) # L2 关图
        except (VectorDBError, EmbedError):
            metrics.degradation_event("L2_to_L3")
        try:    return await self._l3_bm25_mini(query, tenant) # L3 BM25+mini
        except Exception:
            metrics.degradation_event("L3_to_L4")
    # L4 兜底——永远有 answer 返回
    if cached := await self.cache.lookup_loose(query):
        return {"**cached", "degradation_level": "L4_cache"}
    return {"answer": "目前服务繁忙,已为您预约人工客服。",
            "degradation_level": "L4_fallback", "hitl_triggered": True}
```

老司机解读

@circuit 熔断 · 同类错连续 5 次自动断,不让整链被慢服务拖死

degradation_event · 每次降级打 metric,Week 12 面板能看"系统现在跑在哪一级"

每一级都有 answer 返回 · 关键中的关键,不抛 500 给用户白屏

hitl_triggered · 极端情况转 Week 10 的 HITL,系统坏了也让客服接上

2026 提醒 · L1 换模型这段,直接交给 LiteLLM router 做——一份配置声明 gpt-4o→sonnet→haiku 的 fallback 链,不用自己写

MULTI-LEVEL CACHE

AI 系统的 4 级缓存——不只降成本，更是韧性的"第二防线"

上游挂了/模型 timeout 时，缓存让高频问题还有响应，用户感受不到事故

缓存级别	位置	内容	命中率	主要价值
L1 · Browser	CDN / Edge	完整 response	5-10%	0ms 响应
L2 · App	Redis	语义级 response (W15 L1)	20-35%	降成本 + 韧性
L3 · Component	Local LRU	中间结果 (embed/rerank)	40-60%	降 component 成本
L4 · Cold	Object Storage	完整 trace + bad case	N/A	韧性 + 复盘

关键判断

4 级缓存合起来,全链路命中率能到 50-70%。这一半以上的流量,既省了成本、又在上游故障时兜住了底——缓存是我见过唯一一个"降成本"和"提韧性"两件事一起干成的工程。关键是"分层失效",不同级别失效条件完全不同,这也是下一页要讲的精细化失效。

CACHE INVALIDATION

缓存失效——release_id 驱动的精细化失效，命中率再提 30-50%

不是"想清就清"，是"按变化的字段类型精准失效"

```
# services/cache/invalidation.py
class CacheInvalidator:
    """基于 Week 14 release manifest 的精细化失效"""
    def on_release_published(self, new, old):
        if new["spec"]["data"] != old["spec"]["data"]:
            self.cache.delete_pattern("rag:*") # 数据变→失效所有答案
        if new["spec"]["prompt"] != old["spec"]["prompt"]:
            self.cache.delete_pattern("rag:answer:*") # Prompt 变→只失效答案
            # 不删 embedding——它跟 prompt 无关,白留着复用
        if new["spec"]["model"] != old["spec"]["model"]:
            self.cache.delete_pattern("rag:*")
            self.cache.delete_pattern("rerank:*") # 模型变→连 rerank 一起

    def on_data_change(self, lakefs_event): # 监听 Week 14 lakeFS
        for doc in lakefs_event["changed_paths"]:
            self.cache.delete_pattern(f"rag:doc:{doc}:*") # 只删受影响 chunk
```

老司机解读

基于 Week 14 manifest 字段对比 · 不是"全删",是"按变了什么字段失效什么"

Prompt 变了不删 embedding · 因为 embedding 是"数据 + embedder"的函数,跟 prompt 无关

on_data_change 监听 lakeFS · 数据变只删受影响 chunk,不是全清

精细化 vs 粗暴全清 · 命中率能高 30-50%,长期 ROI 极大

一句话 · 缓存失效本身也是工程对象,不是"想清就清"——这是新手和老手的分水岭

RATE LIMIT + QUEUE

限流 + 优雅排队——Edge 层第一道防御，令牌桶最适合 AI

流量 10 倍突增（营销/病毒事件）没限流就被打垮，令牌桶允许短时突发、长期均匀

```
# services/ratelimit.py —— 令牌桶 + 优先级队列
class RAGRateLimiter:
    def __init__(self):
        self.vip = RateLimiter(rate=300, capacity=1000) # VIP 高配额
        self.default = RateLimiter(rate=100, capacity=300)

    async def acquire(self, req) -> bool:
        lim = self.vip if self._tier(req) == "vip" else self.default
        if await lim.try_acquire(): # 1. 有令牌立即过
            return True
        try: # 2. 排队最多等 5s
            await asyncio.wait_for(lim.acquire(), timeout=5.0)
            return True
        except asyncio.TimeoutError:
            return False # 3. 超时转优雅降级

    async def rate_limited_serve(req):
        if not await limiter.acquire(req): # 不让用户看到 429
            return {"answer": "目前服务繁忙,请稍后再试。",
                    "retry_after_seconds": 30, "degradation_level": "L4_rate_limited"}
        return await rag_service.serve(req)
```

老司机解读

VIP 高配额 + Default 普通 · 业务分层,营销/病毒流量来了不影响付费客户

try_acquire 立即返回 + acquire 等 5s · 短时突发能排队,长时排队转降级

"answer" 字段就有友好提示 · 不让用户看到 429 技术错误,完全屏蔽

retry_after_seconds · 给客户端清晰节奏,让重试不打爆

degradation_level 进 Week 12 trace · 限流也是可观测事件,能复盘

INDUSTRY 2026

AI 系统韧性——工具链早已成熟，但有个旧认知要纠正

关键不是工具，是建立"韧性优先"的工程文化

来源	做法	2026 建议
Resilience4j / Sentinel	熔断/降级/隔离/限流四件套	注意:Netflix Hystrix 已退役,别再用
LiteLLM Router	模型 fallback + 负载均衡 + 重试	AI 层降级链交给它,一份配置搞定
Cloudflare AI Gateway	限流+缓存+fallback+监控一体	不想自建的团队,开箱即用
Anthropic 降级实践	opus→sonnet→haiku→cached	官方推荐分层降级,和本课 5 级一致

老司机说

先纠正一个很多老 PPT 还在讲的错:Netflix 的 Hystrix 早就进维护模式、官方都不推荐用了,现在 Java 生态是 Resilience4j,阿里系是 Sentinel。到了 AI 层,2026 你根本不用自己写降级链——LiteLLM router 一份配置声明 opus→sonnet→haiku→cached 的 fallback,负载均衡、重试、超时全带了。

所以我常说:韧性这事,工具早不是瓶颈,瓶颈是团队愿不愿意"上线前就把降级路径想清楚",而不是出了事再抓瞎。

LESSON 03 · SLO · THE CONTRACT

SLO 体系：把"质量目标"变成可量化、可承诺、可执行的工程契约

上线 6 个月后必然发生一件事——业务方问"系统现在到底好不好"

真实场景 · 业务方问"好不好", 你怎么答

你上线半年,老板某天在会上问:"我们这系统现在到底好不好?" 没 SLO 的团队怎么答:"最近指标还可以,CSAT 有 4.2 吧"——含糊、主观、没法问责,老板听完也不踏实。有 SLO 的团队怎么答:"过去 30 天 SLO 达成 99.4%,错误预算还剩 35%"——精确、客观、可执行,老板一听就懂、就放心。这一讲我带你把 Week 11 的评测 + Week 12 的可观测,收口成业务方能拍板签字的 SLO 契约,让 AI 团队不再被"好不好"这种模糊问题反复追问。

咱们这节聊 4 件事:

- 为什么 SLO 和 KPI 是完全不同的工程对象——但 90% 团队混着用
- Copilot 必须有的 6 大 SLO 维度——含传统 SRE 没有的"AI 质量类"
- 为什么"错误预算"是 SLO 的灵魂——没预算的 SLO 就是 KPI
- SLO 阈值怎么从历史数据反推、契约怎么三方签字、2026 怎么做 SLO as Code

SLO IS A CONTRACT

SLO 不是"目标数字"——是工程团队 vs 业务方的"双向契约"

把 SLO 当 KPI，工程团队就永远疲于奔命；SLO 既解放工程团队，也约束业务方

维度	KPI(错配)	SLO(正确)
本质	业务方追求目标	双方契约
谁定	业务方拍	业务 + 工程双签
上限	没有,"越高越好"	有,达到即可
不达标	工程团队加班赶	错误预算耗尽 → 暂停新功能
超额时	"很好,继续"	"用预算做更激进的改动"
对业务方	可以无限提要求	不能高于 SLO 要求

关键判断

我见过太多 AI 团队被 KPI 思维卷死:"P99 再快一点""faithfulness 再高一点",业务方无限加码,工程团队永远在追。SLO 这套 Google SRE 文化的核心,就是给这场追逐画一条线:达到即可、超了就用预算做激进改动、预算烧完就暂停加需求。

SIX SLO DIMENSIONS

Copilot 的 6 大 SLO 维度——AI 系统特有

传统 SRE 只有可用性和延迟, AI 系统还得有"质量类"和"合规类"SLO

传统类

- 可用性 99.5%
- P99 端到端 < 3s
- P50 < 1s
- 传统 SRE 标准
- 错误预算:5%

AI 质量类

- Faithfulness 在线抽样 > 0.85
- Citation Coverage = 100%
- Refusal Rate < 5%
- AI 时代独有
- 错误预算:15%

合规 + 成本

- PII 泄露 = 0(零容忍)
- 合规拦截率 > 99%
- cost/query < \$0.05
- 监管 + 财务要求
- 预算:0%红线 / 20%

老司机说

这 6 个维度里,中间那列"AI 质量类"是关键——传统系统写进去就是写进去了,不用测"对不对";AI 系统会一本正经胡说八道,所以必须把 faithfulness、引用覆盖、拒答率这些做成 SLO。很多团队的 SLO 只抄了传统那套可用性延迟,漏了质量类,等于给一个会撒谎的系统只测了"它在不在线"。

HOW TO SET SLO

SLO 阈值怎么定——从历史数据反推，不是拍脑袋"99.99%"

6 步缺一不可，跳过任何一步 SLO 都会变成"工程团队被卷死"的 KPI

步骤	具体做法	工具支持
1. 收集历史	过去 30-90 天实际表现	Week 12 dashboard
2. 算分位	P50 / P95 / P99 现状基线	Prometheus / Datadog
3. 业务表态	能接受最低多少 / 期望多少	业务方访谈
4. 取交集	现状 + 业务可接受,取 70-80%	会议讨论
5. 试运行	30 天观察是否合理	正式签之前
6. Sign-off	工程 + 业务 + 合规三方签	形成契约文档

踩坑提醒

最容易翻车的是第 1 步和第 4 步——很多人跳过历史数据直接拍一个"99.99%",听着漂亮,结果工程团队为了那多出来的 0.09% 累死累活,业务根本感知不到。SLO 一定从"业务能接受多差"反推,不是从"我想多好"出发。

ERROR BUDGET POLICY

错误预算 + Burn Rate——SLO 的灵魂，快慢双烧

预算没烧完时有权拒绝"再加一点质量而牺牲速度"的请求；烧太快自动冻结

```
# observability/slo/budget_policy.yaml
slos:
  - name: availability
    target: 0.995      # 月预算 = 30天 × 0.005 = 3.6h
    burn_rate_alerts:
      - {name: fast_burn, threshold: 14.4, for: 5m, severity: P1} # 6h 烧光
      - {name: slow_burn, threshold: 6, for: 1h, severity: P2} # 3天 烧光
  - name: ai_quality_faithfulness
    target: 0.85      # 7 天能容忍 15% 低质
    burn_rate_alerts:
      - {name: quality_burn, threshold: 5, for: 30m, severity: P2}
  - name: pii_leak
    target: 0.0      # 零容忍
    burn_rate_alerts:
      - {threshold: 0.0001, for: 1m, severity: P0, action: [page, freeze]}

budget_policies:      # 预算驱动的自动动作
  - {when: "budget_remaining < 25%", action: freeze_non_critical_changes}
  - {when: "budget_remaining < 10%", action: freeze_all_changes}
  - {when: "budget_remaining > 50%", action: green_light_aggressive_changes}
```

老司机解读

fast_burn 14.4x + slow_burn 6x · Google SRE 经典,既抓突发又抓慢退化(Week 12 学过)

budget < 25% 自动冻结非关键变更 · 防止"预算燃尽还在加新功能"

budget > 50% 反过来鼓励激进改动 · 不让团队过度保守,预算是用来花的

PII 零容忍 · 一例触发 page + freeze,合规红线无差别处理

这套让"质量"和"速度"动态平衡 · 不再靠工程师"凭感觉"判断现在该不该停手

SLO CONTRACT DOC

SLO 契约文档——三方签字，让 AI 从"实验"升级为"商业服务"

必须是业务方能理解、能签字的商业承诺，不能只是工程内部文档

```
# docs/slo/omnisupport-copilot-slo-v1.md
## OmniSupport Copilot 服务等级协议 (SLA) v1.0 · 生效 2026-06-01
签字方: 工程 alice@platform · 业务 bob@cs-director · 合规 carol@compliance
```

```
## 2. 服务等级目标 (SLO)
可用性: 月度成功响应率 >= 99.5% (错误预算 3.6h/月)
性能: P50 <= 1.0s | P99 <= 3.0s (99% 请求达标)
AI 质量: 7日 Faithfulness >= 0.85 | Citation 100% | Refusal <= 5%
合规: PII 泄露 = 0 | 合规拦截率 >= 99%
成本: 月度 <= $50,000 | 单次查询 <= $0.05 (中位数)
```

```
## 3. 错误预算管理
预算剩余 < 25% → 冻结非关键变更; < 10% → 冻结所有 + incident review
```

```
## 5. 责任与赔偿
SLO 未达: 工程方 30 天内给改进计划; 业务方不可同期要求新功能
P0/P1 事故: 48 小时内完整 postmortem
```

```
## 6. 变更管理: SLA 修订需三方签字 + 30 天通知期
```

老司机解读

三方签字(工程/业务/合规) · 把 SLO 从"内部约定"升级为"商业契约"

错误预算管理章节 · 显式声明"预算耗尽=冻结变更",质量保护有据可依

责任与赔偿双向约束 · 业务方"不能同期要求新功能",防止"预算烧完还要赶"

变更 30 天通知期 · SLA 不能朝令夕改,契约才稳定

这份文档,就是让 Copilot 从"AI 实验"变成"能对外承诺的商业服务"的那一纸——面试时能拿出来是加分项

INDUSTRY 2026

SLO 文化 = AI 团队工程成熟度的标志——2026 做成 SLO as Code

没 SLO = 还在"靠 oncall 经验"运营; 有完整 SLO = 工程成熟度 L4+

来源	做法	给你的启示
Google SRE	错误预算驱动的工程文化	"质量"和"速度"动态平衡的圣经
OpenSLO + Sloth / Pyrra	SLO 定义写成 yaml → 自动生成告警	SLO as Code,进 Git、可 review
Nobl9	SLO 平台化 + 多数据源	中大型团队不想自建的选择
Anthropic RSP	把 AI 质量 SLO 做成强制项	模型 release 必须满足,业界标准

老司机说

2026 的做法是"SLO as Code"——别再把 SLO 写在 Confluence 里当摆设。用 OpenSLO 这套规范把 SLO 定义写成 yaml,Sloth 或 Pyrra 自动帮你生成 Prometheus 的 burn rate 告警规则,整个 SLO 进 Git、能 review、能版本化,跟你 Week 14 学的治理一脉相承。我的判断很直接:一个 AI 团队有没有完整 SLO 体系,是它工程成熟度的分水岭——有,说明它在"用工程运营";没有,说明它还在"靠老师傅的经验和运气"。



LESSON 04 · OPERATIONAL RUNBOOK

应急 Runbook 体系：让团队从"靠英雄"变成"靠流程"

Week 9 讲过单个 Runbook 怎么写——今天讲 Runbook 体系，外加 2026 的 Agentic SRE

真实场景 · 凌晨 3 点，是"打电话找老 X"还是"打开索引"

判断一个 AI 团队成不成熟，我有个最简单的测试：周末凌晨 3 点出故障，oncall 第一反应是什么？不成熟的团队——"赶紧打电话把老 X 叫起来"；成熟的团队——"打开故障分类树，5 秒找到对应 Runbook"。前者我见得太多了：所有应急经验都锁在那个做了三年的老师傅脑子里，他一离职、一请假、一生病，整个团队就失能。一个生产系统会遇到 50 多种异常，靠人记根本记不住。这一讲我教你把零散 Runbook 组装成体系的应急手册，让新人也能稳定处理，再看看 2026 的 Agentic SRE 怎么把这件事又往前推了一大步。

咱们这节聊 4 件事：

- 为什么"Runbook 写了没人看"是 90% 团队的现状——索引和触发的工程问题
- Runbook 4 级成熟度 + 5 大分类——升到 L3 可执行是 AI 团队的工程拐点
- 从 Bad Case 自动生成 Runbook + Wheel of Misfortune 演练——让应急变成团队复利
- 2026 头条：Agentic SRE——自主 AI 把 MTTR 砍 70%，以及老司机的清醒

A SYSTEM PROPERTY

应急能力是"工程系统的属性"——不是"个人英雄的属性"

老 X 离职/休假/病了 = 团队失能，这是"个人英雄团队"的死穴

等级	具体表现	应急依赖
L0 · 无	"打电话找老 X"	个人英雄
L1 · 散文式	Confluence 几篇文档,找不到	文档 + 经验
L2 · 索引化	故障分类树 + Runbook 库	体系化文档
L3 · 可执行	Runbook 是 Skill Pack (W9) + CLI	工程化执行
L4 · 自主	Agentic SRE 自动诊断/修复(2026)	系统自治 + 人守红线

关键判断

大部分团队卡在 L1——写了一堆文档,散落在 Confluence,出事根本找不到。升到 L3 可执行,需要的不是工具,是"应急工程化"的文化。而 2026 的 L4 已经不是简单脚本自动化了,是 Agentic SRE 自主诊断修复——这一节末尾细讲。



RUNBOOK CATEGORIES

Runbook 必须分类——5 大类、20-30 个常见场景

不分类 = 出事找不到，等于没写

Infra · 基础设施

- pgvector OOM
- Redis 连接断
- Object Storage 慢
- GPU 集群挂
- 占比:30%

Data + Model

- lakeFS 分支冲突
- 索引重建中
- LLM API 429
- Embedding 超时
- 占比:35%

Service + Compliance

- RAG 服务挂
- Agent 工具失败
- HITL 超时积压
- PII 泄露报警
- 占比:35%

老司机说

分类的意义,是让 Week 12 的告警能"自动匹配"到对应 Runbook——oncall 收到告警,下面直接附着该跑哪个 Runbook 的链接,不用他自己在 Wiki 里翻。我的经验:先把最高频的这 25-30 个场景覆盖了,就能解决 80% 的半夜起床。别追求一次写全 50 个,先覆盖高频的。

RUNBOOK INDEX

Runbook 索引——故障分类树，让告警自动匹配到手册

symptoms 字段让 Week 12 告警自动找到对应 Runbook，oncall 不用自己翻

```
# runbooks/index.yaml
categories:
- id: infra
  runbooks:
  - id: RB-INFRA-001
    name: pgvector OOM / 慢查询
    symptoms: [pgvector_p99>5s, OOM_kill] # 告警据此自动匹配
    sla: "P1 → 15min ack, 1h resolve"
    owner: data-platform
    skill_pack: skills/infra-pgvector-recovery # 升级为可执行 (W9)
  - id: model
    runbooks:
    - id: RB-MODEL-001
      name: OpenAI 429 限流
      symptoms: [openai_429, llm_timeout_rate>5%]
      skill_pack: skills/model-fallback-cascade
# 告警 → Runbook 自动关联
alert_to_runbook:
- {alert_name: pgvector_p99_high, runbook_id: RB-INFRA-001, auto_link: true}
- {alert_name: faithfulness_burn, runbook_id: RB-MODEL-002, auto_link: true}
```

老司机解读

symptoms 字段 · 让 Week 12 告警系统"自动匹配"到 Runbook, 不让 oncall 自己找

sla 显式声明 · P0/P1/P2 不同响应时长, 优先级清晰

owner 字段 · 责任清晰, 新人知道找谁问

skill_pack · 把 Runbook 升级为 Week 9 的可执行 Skill——人能跑, Agent 也能跑

auto_link · 告警通知里自动附 Runbook 链接, oncall 不用再打开 Wiki 翻找

BAD CASE → RUNBOOK

从 Bad Case 自动生成 Runbook——让事故变成团队复利

每次复盘自动产出 Runbook 模板 + Skill Pack, "踩过的坑"变成"应急资产"

```
# tools/runbook_from_postmortem.py
def generate_runbook(postmortem_path) -> dict:
    """从 Week 12 postmortem 自动生成 Runbook 草稿"""
    pm = parse_postmortem(postmortem_path)
    runbook = {
        "id": f"RB-AUTO-{pm['incident_id']}",
        "symptoms": pm["detection"]["alert_signatures"], # 复盘的告警特征
        "category": classify_category(pm["root_cause"]["layer"]),
        "sla": determine_sla(pm["severity"]),
        "diagnosis_steps": [s for s in pm["timeline"]["diagnosis_steps"]],
        "fix_steps": [s for s in pm["timeline"]["fix_steps"]],
        "rollback": pm["resolution"]["rollback_steps"],
        "drill_schedule": "每季度演练",
    }
    add_to_runbook_index(runbook) # 入索引
    write_runbook_md(runbook) # 生成文档
    create_skill_pack_from_runbook(runbook) # 生成可执行 Skill (W9)
    return runbook
# $ omni postmortem to-runbook INC-2026-0518-001
# ✓ Runbook + Skill Pack 生成, 演练已排期 2026-08-18
```

老司机解读

从 postmortem 提取字段 · 复盘和 Runbook 一份写两次,工程师不用"再写一遍"

classify_category 自动归类 · 新 Runbook 自动进 5 大类索引,体系化维护

drill_schedule · 每个 Runbook 自动排演练,不让"写了不演"

create_skill_pack · Week 9 学过,Runbook 升级为可执行 Skill,Agent 也能跑

这条复利链 · 每次事故都"增值团队资产",越用越富——这是 SRE 文化的精髓

WHEEL OF MISFORTUNE

Wheel of Misfortune 演练——让 Runbook 永远"活"在肌肉记忆里

写 Runbook 是第一步，真正的考验是"演练"；Google SRE 经典机制

维度	具体做法	工程价值
频率	每周 1 次(30 分钟演练)	不让团队懈怠
随机	随机抽 1 个 oncall + 1 个 Runbook	人人都要练,不让老 X 独享经验
真实	模拟真实告警 + 监控数据	演练贴近实际
评分	主持人按 Runbook 评 1-5 分	检验 Runbook 质量
迭代	演练发现漏洞 → 立即 PR 修	命令过期/key 失效当场修,Runbook 是"活的"

关键判断

这套机制的价值,是让"应急能力"成为"持续训练的肌肉",不会因为半年没出事故就退化。我特别强调"演练发现漏洞立即修"这条——我见过太多 Runbook,真出事时一执行,发现里面的命令早就过期了、备用 key 没更新。没演练过的 Runbook,基本等于没有。写十份不演,不如写三份周周演。

AGENTIC SRE · 2026

2026 头条：Agentic SRE——自主 AI 把 detect→diagnose→remediate 跑起来

这是原来"L4 自动化"在 2026 的真实形态，但人依然守着红线

厂商 / 信号	能力	2026 数据
Resolve.ai	自主事故响应(前 Splunk/OTel 创始团队)	2025-12 独角兽 \$1B,瞄准 80% 自主解决
Cleric	7×24 自动排查告警 + 根因分析	Gartner Cool Vendor 2025(AI for SRE)
Traversal	因果 ML 做根因定位	90%+ 准确率,给 DigitalOcean 省 3.6 万工时/年
行业整体	AI SRE 自动化事故响应	MTTR 砍最高 70%,63% 开发者已在用

老司机解读

2026 这波 Agentic SRE 是真的猛——自主 agent 24 小时盯告警,自己查日志、自己做根因分析,甚至自己按 Runbook 执行修复,把 MTTR 砍掉一大半。而且你注意到没有:它们"学习"的养料,正是你这一节做的东西——结构化的 Runbook、Skill Pack、postmortem。你把这些做扎实了,就是在给未来的 AI SRE 喂饭。

但我要给你泼盆清醒的冷水:再自主的 SRE agent,也不能让它自己决定"要不要回滚生产""要不要冻结服务""要不要动客户数据"——这些高风险动作,必须留一道人工审批(还记得 Week 10 的 HITL 吗)。2026 正确的姿势是:agent 干脏活累活(查、诊断、给方案),人守判断和红线。别把命门交给一个还会犯错的 agent。

LESSON 05 · CAPSTONE · GRADUATION

Capstone 课程毕业：把 15 周收口成可交付、可演讲、可上线的作品

15 周走完了——从 Week 2 的数据契约到 Week 14 的版本治理，今天是真正的毕业典礼

真实场景 · 面试官只会问一句“给我看你做过的东西”

我面试过上百个 AI 工程师,发现一个残酷的事实:听完课、看完文档、跑过 demo,统统不算数。任何一家公司面试,坐下来第一句几乎都是——“给我看你做过的东西”。这时候,大部分候选人只能干瞪眼,或者掏出一个 toy demo。而你不一样:走完这一讲,你手里会有一份能给陌生人看的实物证据——GitHub 仓库 + 答辩 PPT + 运维文档 + 合规白皮书 + 事故复盘 + 12 个月路线图。这六件套合起来,就是任何面试官都能直接评估的“AI 数据工程实战能力证明”。这一讲,咱们一起把它攒齐。

咱们这节聊 4 件事:

- 为什么“15 周学完没作品”是大多数课程的失败——学过 ≠ 拥有

- Capstone 作品集的 6 类核心产物——可演讲、可演示、可复用

- 一份 15 分钟答辩 PPT 的标准 12 页结构——团队/求职/客户都能用

- 毕业后 12 个月演进路线 + 2026 该深扎的 5 个方向

PROVE IT TO A STRANGER

毕业不是"学完"——是"能向陌生人证明你能做"

听完 15 周 ≠ 会做企业 AI 数据工程; Capstone 就是那份"实物证据"

Code + Docs

- GitHub 项目(OmniSupport)
- README + 架构图
- 15 周代码 + 测试
- 让人能"clone 起来跑通"
- 面试/求职用

Engineering Artifacts

- Release Manifest 集合
- SLO 契约 + 评测报告
- Runbook 库 + 演练记录
- 让人看到"工程化能力"
- 团队/客户 sign-off 用

Presentation

- 15 分钟答辩 PPT
- 合规白皮书(W14)
- 12 个月演进路线图
- 让人理解"你的判断"
- 高级面试/提案用

老司机说

我给你交个底:面试时能亮出中间那列"工程化产物"(release manifest、SLO 契约、Runbook 库)的候选人,我见过的不到 20%。大部分人只有一个能跑的 demo。而治理、合规、可观测这些,恰恰是把"会调 API 的"和"能负责一个生产系统的"区分开的东西。你有,就是碾压性优势。

15 WEEKS · YOUR ARSENAL

Week 1-15 工程对象总览——你已经攒下的"工程武器库"

每一周都有具体可交付的工程产物，这张表就是你的"成果地图"

Week	主题	产物
1-2	业务边界 + 数据契约	YAML contract + manifest + risk map
3-4	采集 + Lakehouse	Ingestion pipeline + Iceberg tables
5-6	语义层 + 资产化编排	dbt models + Dagster assets
7-8	非结构化 + RAG 服务化	Doc pipeline + Hybrid retrieval + Rerank
9-10	Agent Skills + 受控 Agent	Skill Pack + Tool contracts + HITL
11-12	评测 + 可观测	RAGAS + LLM Judge + OTel + Dashboards
13-14	GraphRAG + 版本治理	Graph schema + lakeFS + Release manifest
15	上线收官	Cost + SLO + Runbook + Capstone

关键判断

这张表你可以直接截图当"技能清单"——15 周不是听了 15 次课,是攒了 15 类能拿出来的工程产物。

CAPSTONE DECK

15 分钟答辩 PPT 的标准 12 页结构——先讲问题，别先 show 技术

听众不关心你用了 RAG，关心"它解决了什么、达到了什么效果"

Capstone 答辩 PPT 模板 (12 页)

- 1 封面: 项目名 + 你的名字 + 课程结业
- 2 问题陈述: 客户真实业务挑战 ("AI 时代客服该长什么样")
- 3 系统架构图: 数据→检索→生成→Agent→治理, 一图 30 秒看懂
- 4 核心工程承诺: 答得稳(W8) / 办得对(W10) / 可观测(W12) / 可治理(W14)
- 5 Demo · Happy Path: 用户问 → 答案 → 引用 → 工单更新
- 6 Demo · HITL Path: 高风险动作 → 审批 → 执行
- 7 Demo · Bad Case 复盘: 客户报问题 → trace → 修复 → 反例库
- 8 评测数据: 6 类 RAGAS 指标 + 业务 SLO + A/B 对比图
- 9 治理: 一次发布 = 6 类对象原子绑定 + 秒级回滚演示
- 10 合规: EU AI Act / 中国备案 · 一键生成不可篡改白皮书
- 11 关键学习 + 演进路线: "学到的 3 件事" + "下一步 3 件事"
- 12 致谢 + 问答

老司机解读

Slide 2 先讲问题 · 别一上来 show 技术, 听众不关心 RAG, 关心它解决了什么

Slide 5-7 三个 Demo · Happy/HITL/Bad Case, 覆盖最常被问的 3 类场景

Slide 8 用数据说话 · "Faithfulness 0.91" 比 "答得很准" 有说服力 10 倍

Slide 9-10 治理 + 合规 · 这两点 80% 候选人都没, 你有就是绝对优势

Slide 11 演进路线 · 让面试官看到你"持续学习", 不是"做完就完"

12-MONTH ROADMAP

毕业后 12 个月演进路线——课程结束才是真正的开始

光会"做项目"不够，还要持续演进；走完这条线的人会是 AI 时代的领跑者

阶段	目标	关键动作	判断成功
月 1-3 · 巩固	把 15 周代码跑到 100%	复现 OmniSupport + 部署	能给陌生人 demo 不卡
月 4-6 · 拓展	深入 2-3 个技术专题	graph / 评测 / 可观测 深扎	写 3-5 篇技术博客
月 7-9 · 实战	在企业里真落地一次	推动公司/实习一次真上线	客户用 + 评测达标
月 10-12 · 输出	建立行业影响力	开源 / 演讲 / 写课	GitHub 100+ star

老司机说

这条路线不是"必须",但我跟你交心一句:走完的人和不走的人,一年后差距是天壤之别。大部分人学完就把代码扔了,半年后啥也不记得;而认真走完这四阶段的,一年后知识、实践、影响力三位一体,简历上是"我主导落地过一个企业级 AI 系统 + 有开源和博客背书",这在 2026 的就业市场,是能直接跳档的。课程给你的是"广度和地图",能不能变成"深度和身价",看你接下来这 12 个月。

WHAT TO LEARN NEXT

毕业后该深扎的 5 个方向——2026 版，选 1-2 个

15 周给了你"广度"，现在选方向深扎，从"工程师"升级为"专家"

方向	本课程止于	下一步深扎(2026)
多 Agent / Agentic 编排	单 Agent 受控	Agent 协作 / Workflow 编排 / LangGraph 深水区
评测 & AI 可靠性	RAGAS + 门禁	evals 是护城河 / AI SRE / 在线质量守护
Context Engineering	基础 Prompt + RAG	长上下文 / 记忆 / 检索编排 / prompt caching 优化
模型后训练	只用不训	LoRA / DPO / GRPO——2026 训练成本大降,值得学
AI 安全 / Red Team	合规边界	Prompt 注入 / 越狱 / 对抗评测——需求在暴涨

老司机说

给你一个 2026 的方向判断:如果只让我推一个,我推"评测 & AI 可靠性"——因为模型会越来越便宜、越来越强,真正稀缺、真正是护城河的,是"让 AI 系统在生产里可信赖"的能力,而这恰恰是这门课的主线。agentic、context engineering、后训练也都是好方向,选一两个你真感兴趣的,扎进去做深,别贪多。

EIGHT ENGINEERING PROMISES

AI 工程不是"会调 API"——是"交付可信赖的智能服务"

15 周走完，希望你带走的不是"用过多酷模型"，而是这 8 条工程承诺

工程承诺	来自	工程承诺	来自
数据有契约	Week 2	故障可定位	Week 12
答案有证据	Week 8	版本可回滚	Week 14
行为受控制	Week 10	监管可审计	Week 14 L4
质量可量化	Week 11	长期可运营	Week 15

关键判断

这 8 条合起来,就是你今天真正拥有的能力。AI 工程的本质,从来不是"用了多酷的模型",而是"在真实业务约束下,交付一个可信赖的智能服务"。可信赖,就是这 8 个字撑起来的。别让它们停留在 PPT 里——把它们变成你每天写代码时的工程习惯,这才是这门课真正想给你的东西。



COURSE · 收口

课程毕业——15 周总收获 + 给你的赠言

五个侧面，一套完整的企业级 AI 数据工程能力

- Week 1-7 输入侧:数据契约 + 资产化 + 多模态——让 AI 系统的"输入"可信赖
- Week 8-10 服务侧:RAG + Skill + Agent——让 AI 既"答得稳"又"办得对"
- Week 11-12 保障侧:评测 + 可观测——让"质量"和"故障"可量化、可定位
- Week 13-14 演进侧:GraphRAG + 治理——让 AI 系统能"升级"也能"回滚"
- Week 15 运营侧:成本 + 韧性 + SLO + 应急——让系统能"长期稳定跑"

毕业赠言

记住课程开头那句话——AI 工程的本质,不是用了多酷模型,是交付可信赖的智能服务。把今天学到的,明天就用起来,让 AI 真的在你手上"为业务产生价值"。课程到此结束,但你的旅程才刚开始。毕业愉快!

COURSE · GRADUATION

15 周完整闭环 · 你已经是企业级 AI 数据工程师

从 Week 2 数据契约到 Week 14 版本治理,从 Week 6 资产化工厂到 Week 13 GraphRAG——你已经掌握了 2026 年企业 AI 团队需要的完整工程能力栈。

完整 Capstone 作品集（已 push 至 GitHub）

Source Code

15 周完整代码 + 测试

Release Manifests

原子绑定的发布记录

Eval Reports

RAGAS + 业务 SLO 评测

Runbook Library

5 大类 25+ 应急手册

Compliance Docs

EU AI Act / 中国备案白皮书

Capstone Deck

15 分钟答辩 PPT

Architecture

完整系统架构图 + ADR

Roadmap

毕业后 12 个月演进路线

🎓 毕业愉快

把今天学到的,明天用起来——让 AI 真的为业务产生价值。课程到此结束,你的 AI 数据工程师旅程才刚开始。